- **用户信息**
  - 命令

```
# 设置
git config --global user.name "<用户名>"
git config --global user.email "<邮箱>"

# 查询
git config --global user.name
git config --global user.email
```

  - 说明
    - --local（默认值）：适用于当前仓库
    - --global：适用于用户在其操作系统账户下的所有仓库
    - --system：适用于系统中所有用户的所有仓库

- **初始化本地仓库**
  - 命令

```
git init
```

  - 说明
    - 在需要初始化为Git仓库的目录中执行
    - 初始化后生成一个.git子目录（隐藏文件），用于存储Git的版本控制信息和配置

- **状态**

```
git status # 查看当前分支的状态
```

- **暂存**

```
git add <文件名> # 添加单个文件
git add *.<文件后缀> # 添加所有.txt文件
```

```
git add . # 添加当前目录下的所有改动
```

- **提交**
  - 命令

```
git commit -m "<提交信息>" # 创建新的提交
git commit --amend # 修改上一次提交的提交信息 / 将新内容追加到上一次提交
git rm --cached <文件名> # 取消追踪.gitignore中已被追踪的文件
```

  - 说明
    - git commit --amend会重写提交历史，如果这个提交已经被推送到远程仓库，需要使用git push --force强制推送来改变远程仓库的历史

- **提交历史**
  - 命令

```
# 整体
git log # 查看当前分支的提交历史
git log -n 5 # 查看当前分支指定数量的提交历史
git log --all # 查看所有分支的提交历史
git log --oneline # 将每个提交历史压缩到一行
git log --graph # 以图形方式查看提交历史
git log --author="<用户名>" # 搜索特定用户的提交历史

# 单个文件
git log <文件路径> # 查看指定文件的提交历史
git log -- <文件路径（以-开头）> # 查看指定文件的提交历史（文件路径以-开头）
```

  - 说明
    - 需使用less视图快捷键
    - commit后面的一长串字符是哈希值，可以用来回退、比较等，使用时只需复制其一部分
    - --主要用于明确指示命令行解析器命令参数的结束，并开始接受路径或文件名等参数。当文件路径不以-开头时（不会造成混淆），可以省略

- **变基**
  - 命令

```
git rebase -i HEAD~<n> # 基于前n次提交变基
git rebase -i <哈希值> # 基于指定哈希值的提交变基
```

- 说明
  - 修改历史提交信息
    - 将要修改提交信息的每个提交的pick更改为r，保存退出
    - 逐个修改提交信息，保存退出
  - 将多个历史提交合并为一个
    - 将要合并的每个提交的pick更改为s，保存退出
    - 修改提交信息，保存退出
  - 当前分支的变基操作不会影响其他分支，即使所改变的提交为其他分支共有

- **分支**

```
# 查看
git branch # 查看本地分支
git branch -r # 查看远程分支
git branch -a # 查看本地和远程分支
git branch -av # 查看本地和远程分支，并显示最后一次提交信息

# 操作
git branch <新分支名字> # 从当前分支创建新分支
git branch <新分支名字> <旧分支名字> # 从指定分支创建新分支
git checkout <分支名字> # 切换到指定本地分支
git checkout -b <新分支名字> # 创建新分支并切换到它
git branch -d <分支名字> # 删除本地分支
git branch -D <分支名字> # 强制删除本地分支
```

- **合并分支**

```
git merge <分支名字> # 将指定分支合并到当前分支，自动提交
git merge --no-commit <分支名字> # 将指定分支合并到当前分支，不自动提交
git merge --squash <分支名字> # 将指定分支的更改应用到当前分支
git merge <分支名字1> <分支名字2> ... <分支名字n> # 将多个分支合并到当前分支，自动提交
```

```
git merge --abort # 当合并遇到冲突时，终止当前合并
```

- **合并某次提交**

```
git cherry-pick <commit> # 将指定的某个提交应用到当前分支
git cherry-pick <commit1> <commit2> # 依次应用多个指定的提交
git cherry-pick <commit1>^..<commit2> # 将从<commit1>（含）之后到<commit2>（含）的所有提交逐个挑选
git cherry-pick -n <commit> # 不自动提交，而是把修改内容放入暂存区
git cherry-pick --skip # 冲突时跳过当前cherry-pick的提交
git cherry-pick --continue # 解决冲突后继续cherry-pick过程
git cherry-pick --abort # 放弃cherry-pick操作，恢复到操作前的状态
```

- **比较**

```
git diff # 比较工作区与暂存区的差异
git diff --cached # 比较暂存区与上一次提交的差异
git diff <旧哈希值> <新哈希值> # 比较旧提交到新提交的差异
git diff <分支名字1> <分支名字2> # 比较分支1到分支2的差异
git diff HEAD~<n> HEAD # 比较前n次提交到当前提交的差异
```

- **恢复**

```
# 整体
git reset --hard <哈希值> # 恢复到指定提交记录，丢弃所有更改
git reset <哈希值> # 恢复到指定提交记录，将更改存入工作区
git reset --soft <哈希值> # 恢复到指定提交记录，将更改存入暂存区
git revert <哈希值> # 撤销某次提交记录的更改并提交（不会删除之前的提交记录）

# 单个文件
git checkout <哈希值> <文件路径> # 恢复指定文件到指定提交记录，变更存入暂存区
git checkout <哈希值> -- <文件路径（以-开头）>  # 恢复指定文件到指定提交记录，变更存入暂存区（文件路径以-开头）
git restore --source <哈希值> <文件路径> # 恢复指定文件到指定提交记录，变更存入工作区
```

```
git restore --source <哈希值> -- <文件路径（以-开头）> # 恢复指定文件到指定提交记录，变更存入工作区（文件路径以-开头）
```

- **存储**

```
git stash # 保存工作区和暂存区的更改到新的存储
git stash save "<存储信息>" # 保存工作区和暂存区的更改到新的存储，并附带存储信息
git stash list # 查看存储列表
git stash apply # 应用最近一次存储
git stash apply stash@{<n>} # 应用指定存储
git stash pop # 应用最近一次存储，并删除存储
git stash drop # 删除最近一次存储
git stash drop stash@{<n>} # 删除指定存储
git stash clear # 删除所有存储
```